

Agenda — What We Will Cover Today



PART 1 — Core Concepts & Execution Process

- 01 Introduction to Test Execution
- 02 Objectives of Test Execution
- 03 Prerequisites Before Execution
- 04 Test Execution Workflow (10 Steps)
- 05 Types of Test Execution
- 06 Manual Test Execution
- 07 Automated Test Execution (Selenium/Playwright)
- 08 Test Data During Execution
- 09 Validating Expected vs Actual Results
- 10 Recording Test Results

PART 2 — Advanced Topics & Real-World Application

- 11 Defect Identification & Logging
- 12 Retesting
- 13 Regression Testing
- 14 Execution Metrics & Dashboard
- 15 Entry & Exit Criteria
- 16 Real-World Example: OrangeHRM
- 17 Common Challenges & Mistakes
- 18 Best Practices
- 19 Agile & CI/CD Execution
- 20 Reporting | QA Role | Career Growth



Introduction to Test Execution

What it is, why it matters, and how it fits into the STLC



What is Test Execution?

Test Execution is the phase in the Software Testing Life Cycle (STLC) where test cases are formally run against the application under test (AUT). Testers execute the designed test cases, compare actual outcomes with expected results, and document all findings — including defects, observations, and evidence.

WHAT HAPPENS DURING TEST EXECUTION

- Test cases written in earlier STLC phases are formally executed
- Actual system behavior is observed and compared to expected behavior
- Defects (bugs) are identified, documented, and reported to developers
- Test results are recorded — Pass, Fail, Blocked, or Not Executed
- Retesting and regression testing occur after defect fixes

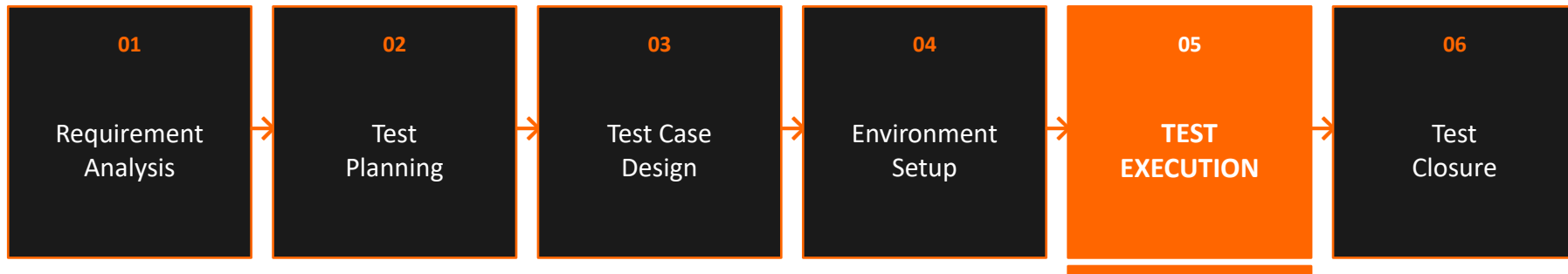
WHY IT IS THE MOST CRITICAL STLC PHASE

- This is where real defects are discovered before reaching end users
- Execution provides objective evidence of software quality
- Results drive go/no-go decisions for software releases
- It converts theoretical test plans into measurable quality data
- All prior STLC phases (planning, design) feed into this phase

Test Execution in the Software Testing Life Cycle (STLC)



Test Execution sits at the center of the STLC — all prior phases prepare for it, and all subsequent phases depend on its results.



▲ YOU ARE HERE

KEY INPUTS INTO TEST EXECUTION

- Test Cases — documented in the Test Case Design phase
- Test Data — positive, negative, and boundary data sets
- Test Environment — configured application + infrastructure

KEY OUTPUTS FROM TEST EXECUTION

- Execution Results — Pass / Fail / Blocked / Not Executed
- Defect Reports — logged in defect tracking tools (JIRA, etc.)
- Execution Reports — metrics, coverage, quality evidence

Why Test Execution is Critical in Quality Engineering



Defect Discovery

The primary mechanism for uncovering hidden defects before they reach production and impact real users.

Quality Evidence

Provides objective, documented proof that the software has been systematically tested against requirements.

Risk Reduction

Identifies functional gaps and behavioral anomalies, reducing the risk of costly post-release failures.

Release Decisions

Execution results directly inform go/no-go release decisions by management, stakeholders, and the business.

Compliance & Audit

For regulated industries, execution evidence is mandatory for compliance audits and certification.

Team Communication

Execution reports communicate software quality status to developers, project managers, and stakeholders.



Objectives of Test Execution

What we aim to achieve by executing test cases



Objectives of Test Execution



01

Validate Application Functionality

Confirm that every feature, function, and module behaves exactly as specified in the requirements documentation.

02

Verify Expected Behavior

Systematically compare the system's actual behavior against pre-defined expected outcomes for every test scenario.

03

Identify Defects Early

Uncover bugs, defects, and inconsistencies before the software is released to end users or production environments.

04

Confirm Requirement Implementation

Verify that all business and technical requirements have been correctly implemented in the system under test.

05

Measure Software Quality

Generate objective metrics — pass rates, defect density, coverage — that quantify the quality level of the software build.



Prerequisites Before Test Execution

What must be ready before you can begin executing test cases



Prerequisites Before Test Execution — What Must Be Ready



Approved Test Cases

All test cases must be reviewed, approved by the QA lead, and placed in 'Ready to Execute' state in the test management tool (e.g., TestRail, Zephyr, JIRA).



Prepared Test Data

Valid, invalid, and boundary test data sets must be available. Data must be prepared in the test environment — not in production.



Stable Test Environment

The application must be deployed to a dedicated test environment. Server, database, network, and all third-party integrations must be operational.



Deployed Application Build

The specific build version to be tested must be deployed. The build number must be recorded in execution reports for traceability.



Access Permissions & Credentials

Testers must have valid login credentials, access to the AUT, and access to test management and defect tracking tools.

Execution Readiness Checklist — Pre-Execution Gate



Before any test execution cycle begins, the QA team must verify each item in this checklist. If any critical item is missing, execution should NOT proceed.

#	Checklist Item	Responsible	Status
1	Test cases written, reviewed, and approved	QA Lead	✓ Ready
2	Test data prepared (positive, negative, boundary)	QA Engineer	✓ Ready
3	Test environment set up and stable	DevOps / QA	✓ Ready
4	Application build deployed to test environment	Developer	✓ Ready
5	Build version number documented	QA Engineer	✓ Ready
6	Tester access credentials confirmed	QA Lead	✓ Ready
7	Test management tool configured (e.g., JIRA/TestRail)	QA Lead	✓ Ready
8	Defect tracking tool accessible	QA Engineer	✓ Ready
9	Entry criteria reviewed and signed off	QA Lead / PM	✓ Ready
10	Risk assessment completed	QA Lead	✓ Ready



Test Execution Workflow

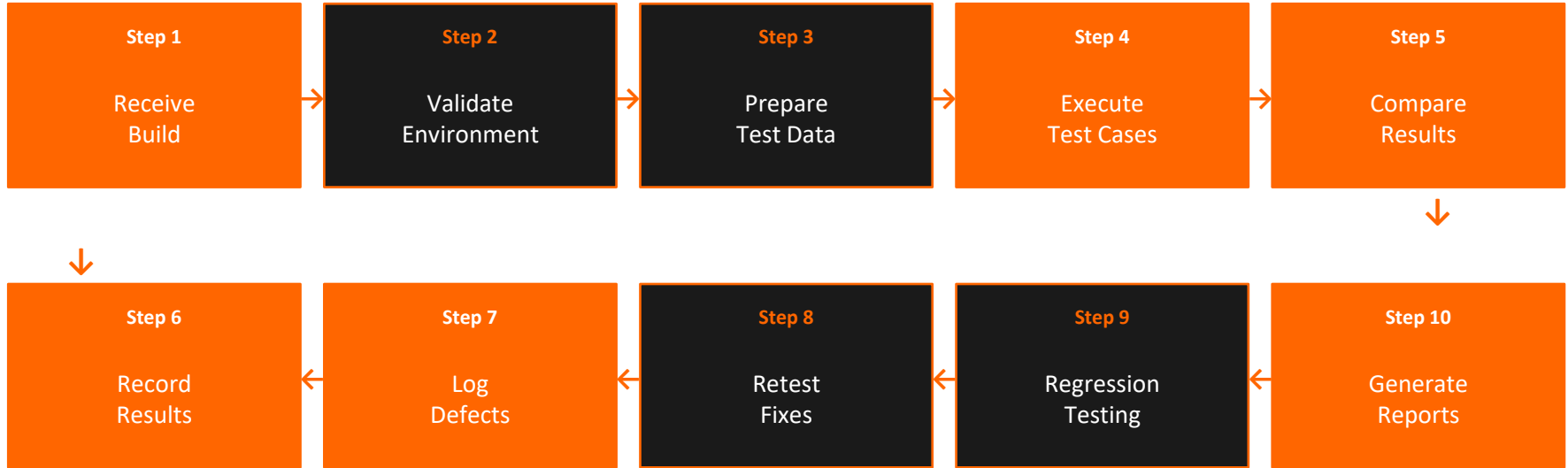
The complete 10-step test execution process from build to report



Test Execution Workflow — Full 10-Step Lifecycle Diagram



Every test execution cycle follows this structured workflow. Each step must be completed before moving to the next.



Test Execution Workflow — Steps 1 to 5 (Detailed)



01

Receive Build

The development team deploys a new application build to the test environment. QA receives the build notification with release notes. The build number, version, and scope of changes are documented before any testing begins.

02

Validate Environment

Testers verify the test environment is fully operational: application is accessible, databases are seeded with correct data, third-party services are connected, and all configurations match the test plan requirements.

03

Prepare Test Data

Appropriate test data is loaded or confirmed in the system. Positive data (valid inputs), negative data (invalid inputs), and boundary values are all prepared. Data from previous cycles is cleaned or reset.

04

Execute Test Cases

Testers systematically execute each test case by following the documented steps. Actions are performed on the AUT exactly as described. No steps are skipped. Both manual and automated test cases are run in this phase.

05

Compare Expected vs Actual Results

For each executed test case, the actual system behavior is compared against the documented expected result. Any deviation — no matter how minor — is flagged for further investigation and potential defect logging.

Test Execution Workflow — Steps 6 to 10 (Detailed)



06

Record Test Results

Each executed test case is marked with a result status: Pass, Fail, Blocked, or Not Executed. Evidence (screenshots, logs) is attached. Results are recorded in the test management tool (TestRail, JIRA Xray, Zephyr) in real time.

07

Log Defects

When a test case fails, a detailed defect report is created in the defect tracking tool (e.g., JIRA). It includes: defect title, steps to reproduce, expected result, actual result, severity, priority, screenshots, and environment details.

08

Retest Fixes

After the development team resolves a defect, QA re-executes the specific failed test case(s) to confirm the fix is working. This is called retesting. Only the failing test cases are re-run — not the entire suite.

09

Perform Regression Testing

After fixes are applied, regression testing ensures that the changes haven't introduced new defects in previously passing areas. Automated regression suites are run to maximize coverage and speed.

10

Generate Reports

After the execution cycle, comprehensive reports are generated: Test Execution Summary Report, Defect Summary, Pass/Fail metrics, Coverage metrics, and Risk status. These are shared with the project manager and stakeholders.



Types of Test Execution

Understanding the different modes and categories of test execution



Types of Test Execution — Overview with Real-World Context



Manual Test Execution

Testers execute test cases step-by-step without automation tools. Ideal for exploratory, UI/UX, and ad-hoc testing.

Automated Test Execution

Test scripts (Selenium, Playwright) execute test cases automatically. Ideal for regression, load, and repetitive scenarios.

Smoke Test Execution

A quick, high-level test of the build to verify core functionality works before full execution begins (sanity check of the build).

Sanity Test Execution

Narrow, focused testing of a specific area after a bug fix. Confirms the fix works without performing full regression testing.

Regression Test Execution

Re-execution of existing test cases after code changes to ensure previously working functionality hasn't been broken.

User Acceptance Testing (UAT)

End users or business stakeholders validate the software meets business needs and real-world requirements before release.



Manual Test Execution

Step-by-step process for executing test cases manually



Manual Test Execution — Step-by-Step Process



1. Open Test Case

Navigate to the assigned test case in your test management tool (TestRail / JIRA / Excel). Read the entire test case including: Title, Prerequisites, Test Steps, Expected Results, and Test Data.

2. Prepare the Environment

Log into the application under test using the credentials specified in the test case. Navigate to the correct module/page. Ensure all prerequisites are met (user roles, data, configuration).

3. Execute Each Step

Follow each test step exactly as documented. Perform the action described (click button, enter data, submit form). Do NOT improvise or skip steps — execution must match the documentation precisely.

4. Observe Actual Results

After each action, observe what the system actually does. Note the system's response: success message, error, page redirect, data displayed, etc. Compare immediately against the expected result.

5. Record Outcome

Mark the test case as PASS if actual = expected. Mark FAIL if a deviation exists. Mark BLOCKED if a prerequisite or prior step prevents execution. Capture screenshots and logs as evidence for all results.

6. Log Defects (if Fail)

For any failed test case, immediately log a defect in JIRA with full reproduction steps, screenshots, severity, priority, and environment details. Link the defect to the test case in the test management tool.

Manual Test Execution — Sample Test Case Walkthrough



SCENARIO: OrangeHRM — Login Module Test Execution (Manual)

Step #	Test Step (Action)	Test Data Used	Expected Result	Actual Result	Status
1	Open browser and navigate to OrangeHRM URL	URL: demo.orangehrm.com	Login page is displayed correctly	Login page displayed	✓ Pass
2	Enter valid Username in the username field	Admin	Field accepts input without error	Input accepted	✓ Pass
3	Enter valid Password in the password field	admin123	Password is masked with asterisks	Password masked	✓ Pass
4	Click the Login button	N/A	User is redirected to the Dashboard	Dashboard displayed	✓ Pass
5	Verify the welcome message on Dashboard	Admin	'Welcome PIM' label visible	Label visible	✓ Pass
6	Enter invalid password and click Login	wrongpass	Error: 'Invalid credentials' is displayed	Error displayed	✓ Pass



Automated Test Execution

Selenium, Playwright, and the role of automation in modern QA



Automated Test Execution — Role, Benefits & Tools



Automation executes pre-written test scripts against the application without human intervention, dramatically increasing speed, repeatability, and coverage.

WHEN TO USE AUTOMATED EXECUTION

- Regression testing — executed after every build/fix
- Repetitive test cases run across multiple environments
- Large data sets requiring data-driven testing
- Smoke tests at the start of every CI/CD pipeline run
- Cross-browser and cross-platform compatibility testing

INDUSTRY TOOLS & FRAMEWORKS

Selenium WebDriver

Industry-standard web automation framework (Java, Python, C#)

Playwright

Modern automation tool by Microsoft with superior speed & reliability

Eclipse IDE

Primary Java IDE for writing Selenium test scripts

IntelliJ IDEA

Advanced IDE supporting Java, Kotlin, and automation frameworks

TestNG / JUnit

Test runners for managing, scheduling & reporting automation tests

AUTOMATION EXECUTION BENEFITS

- Up to 10x faster than manual execution for regression suites
- 100% repeatable — eliminates human error in step execution
- Provides immediate feedback in CI/CD pipelines (fail-fast approach)

Automated Execution — Selenium WebDriver (Java) Example



The following Selenium test case automates the OrangeHRM login flow. This is executed via TestNG in Eclipse IDE or IntelliJ IDEA.

```
01 import org.openqa.selenium.WebDriver;
02 import org.openqa.selenium.chrome.ChromeDriver;
03 import org.openqa.selenium.By;
04 import org.testng.Assert;
05 import org.testng.annotations.Test;
06
07 public class OrangeHRMLoginTest {
08     @Test
09     public void testValidLogin() {
10         WebDriver driver = new ChromeDriver();
11         driver.get("https://demo.orangehrm.com");
12         driver.findElement(By.name("txtUsername")).sendKeys("Admin");
13         driver.findElement(By.name("txtPassword")).sendKeys("admin123");
14         driver.findElement(By.id("btnLogin")).click();
15         String title = driver.getTitle();
16         Assert.assertTrue(title.contains("Dashboard"), "Login Failed!");
17         driver.quit();
18     } // Test result logged automatically by TestNG
19 }
```

Automated Execution — Playwright (JavaScript / Node.js) Example



Playwright by Microsoft is a modern alternative to Selenium. It offers faster execution, auto-wait, and better reliability across Chrome, Firefox, and WebKit.

```
01  const { test, expect } = require('@playwright/test');
02
03  test('OrangeHRM Login — Valid Credentials', async ({ page }) => {
04    await page.goto('https://demo.orangehrm.com');
05
06    // Fill login form
07    await page.fill('[name=txtUsername]', 'Admin');
08    await page.fill('[name=txtPassword]', 'admin123');
09    await page.click('#btnLogin');
10
11    // Assert Dashboard loaded
12    await expect(page).toHaveTitle(/Dashboard/);
13    await expect(page.locator('.welcome-message')).toBeVisible();
14  });
15  // Run: npx playwright test --reporter=html
```

PLAYWRIGHT vs SELENIUM — KEY DIFFERENCES

- Playwright: Built-in auto-wait, no need for explicit waits — Selenium requires manual wait management
- Playwright: Cross-browser support (Chrome, Firefox, WebKit) in one API — Selenium needs separate drivers



Test Data During Test Execution

How test data is prepared, used, and managed during execution



Test Data Usage During Execution — Types, Examples & Best Practices



Test data is the input used during test execution. Using the right data ensures comprehensive coverage — including valid, invalid, and edge-case scenarios.

Positive Data

Valid inputs that should produce a successful result.

EXAMPLE:

Username: Admin | Password: admin123 → Expected: Login Successful

Negative Data

Invalid inputs designed to trigger error messages or validation.

EXAMPLE:

Username: (empty) | Password: wrong → Expected: Error message displayed

Boundary Data

Values at the exact edges of acceptable input ranges.

EXAMPLE:

Password field min 6 chars: enter 5 chars (fail), 6 chars (pass), 7 chars (pass)

Special Characters

Unusual characters that can expose input validation vulnerabilities.

EXAMPLE:

Username: admin<script> → Expected: System rejects or sanitizes input



Validating Expected vs Actual Results

The core comparison at the heart of every test case execution



Expected vs Actual Results — The Core of Test Execution



Every test case has a documented Expected Result. The Actual Result is what the system does when the test is run. Comparing them determines PASS or FAIL.

EXPECTED RESULT

What the test case says SHOULD happen.

Documented during the Test Case Design phase. Written based on business requirements, functional specifications, or user stories. This is the benchmark.

EXAMPLE:

User enters correct credentials and clicks Login →
System should display the Dashboard page with welcome message.

✓ PASS — Actual = Expected

ACTUAL RESULT

What the system ACTUALLY does when the test is executed.

Observed in real time by the tester. Captured with screenshots, logs, or videos. This is the reality — and it may or may not match the expectation.

EXAMPLE:

User enters correct credentials →
System shows an error: "Invalid Credentials" (this is a defect!).

✗ FAIL — Actual ≠ Expected → Log Defect

VS



Recording Test Results

How to document and track execution outcomes accurately



Test Result Statuses — What Each Status Means



PASS

The test case was executed successfully. The actual result matches the expected result. No defect exists for this test case.

ACTION:

Record as Pass. Attach screenshot as evidence. Move to next test case.



FAIL

The actual result does NOT match the expected result. A defect exists. This indicates a bug in the application under test.

ACTION:

Record as Fail. Log a defect in JIRA with full details. Attach screenshots and logs.



BLOCKED

The test case cannot be executed due to an unresolved dependency — a prior defect, missing data, or environment issue prevents execution.

ACTION:

Record as Blocked. Note the blocking reason. Escalate to QA Lead immediately.



NOT EXECUTED

The test case has not been run yet in this execution cycle. It is in the queue waiting to be executed by the QA team.

ACTION:

Track remaining count. Plan execution timeline. Communicate status in daily stand-up.

Sample Test Execution Result Sheet — Real-World Template



This is a standard execution result sheet format used by QA teams. Each row represents one executed test case.

TC ID	Test Case Title	Priority	Test Data	Expected Result	Actual Result	Status	Defect ID
TC-001	Login with valid credentials	High	Admin/admin123	Dashboard displayed	Dashboard displayed	PASS	—
TC-002	Login with invalid password	High	Admin/wrong	Error shown	Error shown	PASS	—
TC-003	Login with empty username	Medium	(blank)/admin	Validation shown	Validation shown	PASS	—
TC-004	Apply for Annual Leave	High	5 days, valid dates	Leave submitted	Error 500 shown	FAIL	BUG-102
TC-005	View Leave Balance	Medium	Admin user	Balance displayed	Session timeout	BLOCKED	BUG-100
TC-006	Export Employee Report	Low	Admin role	PDF downloaded	Not yet executed	N/E	—
TC-007	Reset password via email	High	Valid email	Reset link sent	Reset link sent	PASS	—



Defect Identification During Execution

How testers detect, categorize, and classify defects found during testing



Defect Identification During Execution — Types & Real-World Examples

Functional Defects

The application fails to perform its intended function. A feature does not work as specified in the requirements.

e.g. Clicking 'Submit Leave' does nothing — form does not save to database.

UI / Display Defects

Visual or layout issues that affect user experience but may not break functionality.

e.g. Dashboard title overlaps the navigation bar on 1080p resolution screens.

Data / Calculation Defects

Incorrect data is stored, displayed, or calculated by the system.

e.g. Leave balance shows 15 days remaining when employee has only taken 2 of 20.

Performance Defects

The system is too slow, times out, or fails under normal or expected load conditions.

e.g. Employee search takes 45 seconds to return results — requirement states under 3 seconds.

Security Defects

Security vulnerabilities or access control failures that allow unauthorized access.

e.g. Junior employee can view all payroll data — should only see their own records.

Environment Defects

Defects caused by incorrect environment configuration rather than application code.

e.g. Login works on Chrome but fails on Firefox due to missing SSL certificate on test server.



Defect Logging During Execution

How to capture, document, and report defects professionally in JIRA



Defect Logging Process — Step-by-Step



Confirm the Defect

1

Re-execute the failing test case at least once more to confirm it is a consistent defect and not a one-time environment glitch.

Capture Evidence

2

Take full-page screenshots of the issue. Capture browser console logs, server error logs, network response data, and video recording if possible.

Open a New Defect in JIRA

3

Create a new Issue of type 'Bug' in the project's JIRA board. Fill all mandatory fields — nothing should be left blank.

Write a Clear Defect Title

4

Use the format: [Module] — [What happened]. Example: "Login Module — System returns 500 Error on valid credential submission."

Document Steps to Reproduce

5

List every step a developer needs to reproduce the defect: environment, test data, actions performed, and the exact point of failure.

Assign Severity & Priority

6

Severity: How serious is the impact? (Critical/Major/Minor/Trivial). Priority: How urgently must it be fixed? (High/Medium/Low).

Link to Test Case & Assign

7

Link the defect to the failed test case ID. Assign to the correct developer or team lead. Set correct Fix Version and Sprint.

Sample Defect Report — JIRA Bug Template



The following represents a properly documented defect report as would appear in JIRA. Every field must be completed with precision.

Defect ID	BUG-102
Title	Leave Application Module — System returns HTTP 500 Error on valid Leave submission
Module	Leave Management
Reported By	QA Engineer: Sipho Dlamini
Date Reported	2024-07-15
Severity	Critical — Core business function is completely broken
Priority	High — Must be fixed before next release
Environment	Test Environment Build: v2.4.1 Browser: Chrome 126 OS: Windows 11
Steps to Reproduce	1. Login as Admin 2. Navigate to Leave > Apply 3. Select Leave Type: Annual 4. Select valid date range 5. Click 'Apply' button
Expected Result	Leave request is saved and confirmation message is displayed: 'Leave Applied Successfully'
Actual Result	System returns HTTP 500 Internal Server Error. Leave request is NOT saved. No user-friendly message shown.
Attachments	screenshot_leave_500_error.png console_log_2024-07-15.txt network_HAR_file.json
Assigned To	Backend Developer: Lethabo Mokoena
Status	OPEN — Awaiting Developer Fix



Retesting

Verifying that reported defects have been successfully fixed by developers



Retesting — What, Why, and How



Retesting is the process of re-executing a specific test case that previously FAILED — after the development team claims to have fixed the underlying defect.

RETESTING WORKFLOW — STEP BY STEP

01

Defect Fixed by Developer

Developer resolves the defect and updates the JIRA ticket status to 'Fixed' or 'Ready for Retest'. A new build is deployed to test environment.

02

QA Picks Up Defect for Retest

QA identifies the fixed defect. Retrieves the original failed test case. Confirms the new build is deployed and environment is ready.

03

Execute the Specific Failing Test Case

Re-run ONLY the test case that was marked as FAIL. Follow the exact same steps with the exact same test data. Do not change anything.

04

Compare Results

If Actual Result now matches Expected Result → Mark the test case PASS. Update the JIRA defect status to 'Closed' or 'Verified'.

05

Document Outcome

Record the retest result in the execution sheet. Note the build version in which the fix was confirmed. Add evidence (screenshots) to the JIRA ticket.



Regression Testing During Execution

Ensuring defect fixes haven't broken previously working functionality



Regression Testing — Why It Happens and How It Works

Regression Testing is the execution of a broad set of test cases to verify that recent code changes (fixes, enhancements) have not broken any existing functionality.

WHY REGRESSION HAPPENS AFTER FIXES

- Every code change — even a minor bug fix — can have unintended side effects
- A fix in the Login module could break the Password Reset flow
- Database changes can affect multiple modules simultaneously
- Regression testing prevents delivering a 'fixed' product that is now broken elsewhere

MANUAL vs AUTOMATED REGRESSION

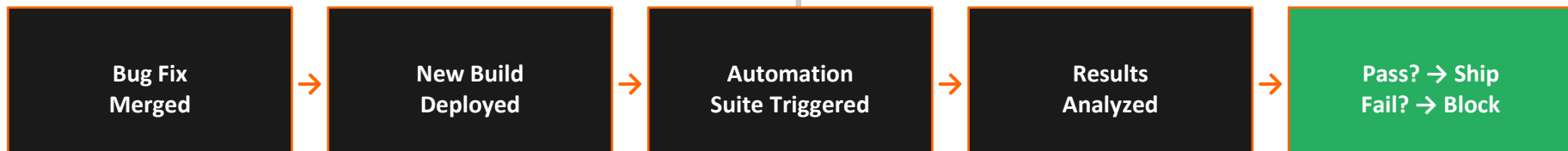
MANUAL REGRESSION

Time-consuming. Prone to human error. Used for complex exploratory scenarios. Suitable for new feature areas without existing automation.

AUTOMATED REGRESSION (Best Practice)

Selenium / Playwright run thousands of tests in minutes. Runs in CI/CD pipelines on every build. Zero human effort after initial script creation.

REGRESSION EXECUTION FLOW





Execution Metrics & Dashboard

Measuring, tracking, and communicating test execution quality



Execution Metrics — What We Measure and Why



Test Execution Progress

Formula: $(\text{Executed TCs} / \text{Total TCs}) \times 100$

$68 / 80 \times 100 = 85\% \text{ Progress}$

Pass Percentage

Formula: $(\text{Passed TCs} / \text{Executed TCs}) \times 100$

$54 / 68 \times 100 = 79.4\% \text{ Pass Rate}$

Fail Percentage

Formula: $(\text{Failed TCs} / \text{Executed TCs}) \times 100$

$10 / 68 \times 100 = 14.7\% \text{ Fail Rate}$

Blocked Percentage

Formula: $(\text{Blocked TCs} / \text{Executed TCs}) \times 100$

$4 / 68 \times 100 = 5.9\% \text{ Blocked Rate}$

Defect Density

Formula: $(\text{Total Defects} / \text{Total TCs Executed}) \times 100$

$12 \text{ defects} / 68 \text{ TCs} = 0.18 \text{ defects/TC}$

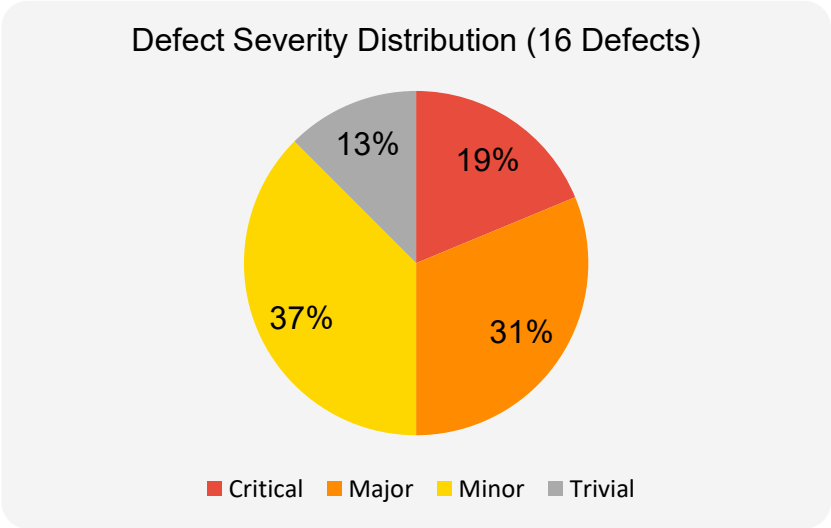
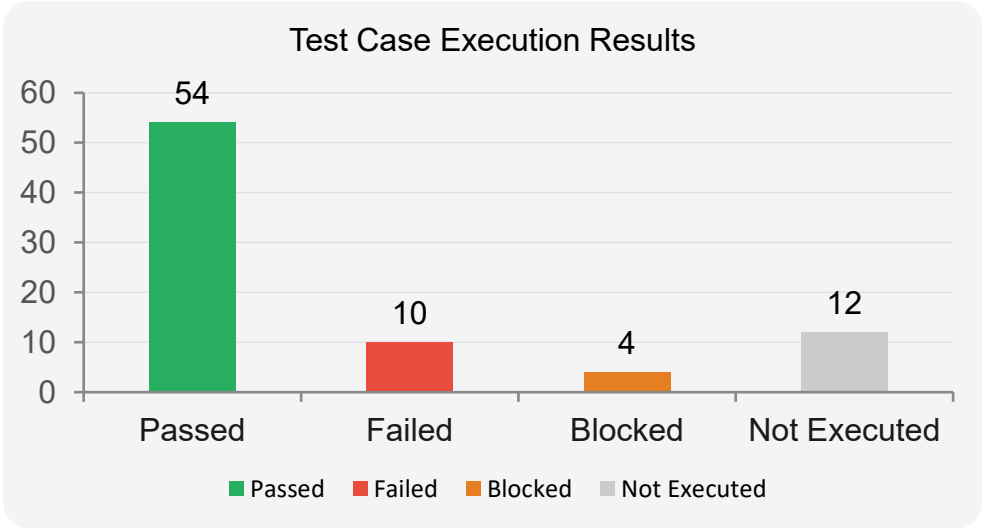
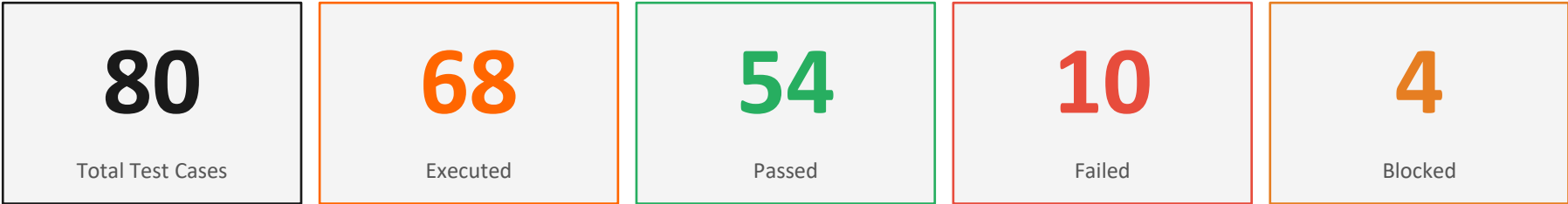
Execution Coverage

Formula: $(\text{Executed TCs} / \text{Requirements}) \times 100$

$68 \text{ TCs cover } 42 / 50 \text{ requirements} = 84\%$

Sample Execution Metrics Dashboard

The following metrics represent a typical mid-sprint execution snapshot shared with the project team and stakeholders.





Entry & Exit Criteria

The formal gates that control when execution starts and when it ends



Entry & Exit Criteria for Test Execution — Formal Quality Gates



Entry and Exit criteria are formal conditions that control the quality of the execution process. Without them, testing becomes uncontrolled and unreliable.

ENTRY CRITERIA — Before Execution Starts

- All test cases written, reviewed, and formally approved
- Test environment fully configured and verified as stable
- Application build deployed with release notes available
- Test data prepared for all test scenarios (positive/negative/boundary)
- Tester access to all required systems confirmed
- Test management and defect tracking tools configured
- Entry criteria reviewed and signed off by QA Lead and PM

EXIT CRITERIA — When Execution Ends

- All planned test cases executed (or a justified subset with sign-off)
- All Critical and High severity defects resolved and retested
- Pass rate meets or exceeds agreed threshold (e.g., 90%)
- No blocker or critical defects remain in OPEN status
- Test Summary Report generated and reviewed by QA Lead
- Defect Summary Report shared with development team
- Stakeholders formally sign off on execution completion

IMPORTANT: If exit criteria cannot be met, the QA Lead must communicate the risk to stakeholders and obtain formal approval before release proceeds.



Real-World Execution

Example: OrangeHRM

Complete end-to-end test execution walkthrough on a real-world HR system



Real-World Example — OrangeHRM: Test Cases & Execution Plan



OrangeHRM is an open-source Human Resource Management system used by organizations globally. We will execute the Login and Leave Application flows.

System: OrangeHRM Demo

Build: v4.7.2 | Chrome 126

Tester: QA Engineer Sipho Dlamini

TC ID	Module	Test Case Title	Priority	Data	Type
TC-001	Login	Valid credentials — Admin user login	High	Admin / admin123	Positive
TC-002	Login	Invalid password — error message check	High	Admin / wrongpass	Negative
TC-003	Login	Empty username — validation check	Medium	(empty) / admin123	Negative
TC-004	Leave	Apply for Annual Leave — valid date range	High	5 days annual leave	Positive
TC-005	Leave	Apply leave with past date — validation	Medium	Date in past	Negative
TC-006	Leave	View Leave Balance — correct count shown	Medium	Admin user	Positive
TC-007	Leave	Cancel pending leave request	Low	Existing pending request	Positive
TC-008	Login	Session timeout after inactivity	Medium	30 min wait	Boundary

Real-World Example — OrangeHRM: Execution Results & Defects Found



After executing all 8 test cases, the following results were recorded. Two defects were identified and logged in JIRA.

TC ID	Test Case Title	Expected	Actual Result	Status	Defect
TC-001	Valid login	Dashboard shown	Dashboard shown	PASS	—
TC-002	Invalid password	Error shown	Error shown	PASS	—
TC-003	Empty username validation	Required msg	Required msg	PASS	—
TC-004	Apply Annual Leave	Confirmation shown	HTTP 500 Error	FAIL	BUG-102
TC-005	Past date validation	Validation error	Validation error	PASS	—
TC-006	View Leave Balance	Balance shown	Session timeout	BLOCKED	BUG-100
TC-007	Cancel leave request	Cancelled	Cancelled	PASS	—
TC-008	Session timeout	Logout redirect	Logout redirect	PASS	—

EXECUTION SUMMARY

8
Total TCs

8
Executed

6
Passed

1
Failed

1
Blocked

2
Defects

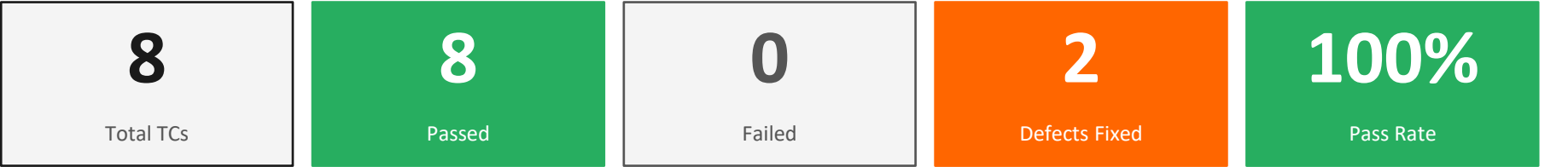
Real-World Example — OrangeHRM: Retesting After Bug Fix



After BUG-102 was fixed by the developer in Build v2.4.2, retesting was performed. BUG-100 (session issue) required environment fix before retest.

Defect ID	Related TC	Defect Description	Fix Build	Retest Result	Status
BUG-102	TC-004	Leave Application returns HTTP 500 Error	v2.4.2	PASS — Fixed	CLOSED
BUG-100	TC-006	Session timeout causes blocked Leave Balance view	v2.4.2	PASS — Fixed	CLOSED

FINAL EXECUTION OUTCOME AFTER RETESTING



KEY LEARNING FROM THIS EXAMPLE

- Real defects (HTTP 500) can only be found during actual execution — not during test case design
- Environment issues (BUG-100) are just as important as application defects to track and fix
- Retesting confirms fixes and moves the cycle forward — documentation at every step is non-negotiable



Common Challenges During Test Execution

Real-world obstacles that QA engineers face and how to handle them



Common Challenges During Test Execution — Causes & Solutions

⚠ Environment Instability

Cause: Test server crashes, database resets, or network failures mid-execution.

Solution: Maintain a dedicated, stable test environment. Always record build/environment version before starting.

⚠ Incorrect or Missing Test Data

Cause: Test data was deleted, modified by another team, or never prepared correctly.

Solution: Use dedicated test data sets. Implement data reset scripts that restore data before each test run.

⚠ Requirement Misunderstandings

Cause: Expected results were documented incorrectly due to ambiguous requirements.

Solution: Always clarify requirements with the BA or PO before writing expected results. Attend sprint grooming sessions.

⚠ Build Failures

Cause: Developer deploys a broken build that prevents any meaningful testing.

Solution: Require smoke test sign-off before QA begins full execution. Use entry criteria gates to block broken builds.

⚠ Flaky Automation Scripts

Cause: Automated tests pass sometimes and fail others due to timing issues or data dependencies.

Solution: Add explicit waits and retry logic. Use Page Object Model for stability. Investigate root causes of every flaky test.

⚠ Scope Creep During Execution

Cause: New features added mid-sprint without corresponding test cases being updated.

Solution: Freeze scope at sprint start. Raise change management flags if scope changes during active execution cycles.



Common Mistakes In Test Execution

Errors that new QA engineers make and how to avoid them



Common Mistakes During Test Execution — Learn From These



✗ Skipping Validation Steps

Rushing through test steps without properly checking the expected result after each action. This leads to missed defects and false Pass results.

✓ Fix: Always validate the outcome of EVERY step — not just the final result. Be thorough, not fast.

✗ Poor Result Documentation

Recording only 'Pass' or 'Fail' without evidence. No screenshots, no actual result description, no test data noted.

✓ Fix: Document everything. Screenshots are mandatory for Fails and Blocks. Actual result must describe what happened.

✗ Weak Defect Reporting

Logging defects with vague descriptions like 'Login not working' without steps, data, or expected vs actual results.

✓ Fix: Follow the defect template strictly. Developers must be able to reproduce the defect from your report alone.

✗ Ignoring Edge Cases

Only testing the 'happy path' (valid inputs). Edge cases, boundary values, and error conditions expose the most critical defects.

✓ Fix: Always test positive, negative, AND boundary scenarios for every feature. Never skip negative testing.

✗ Executing Outdated Test Cases

Running test cases that were never updated to reflect requirement changes, leading to incorrect results and confusion.

✓ Fix: Always verify test case version before execution. Review requirements if test cases seem inconsistent with the AUT.

✗ Not Communicating Blockers

Continuing other tests silently while a blocker exists, delaying the team's awareness of critical environment or defect issues.

✓ Fix: Immediately escalate blockers to the QA Lead. Update the test management tool in real time. Send a Slack/Teams message.



Best Practices in Test Execution

Professional habits that distinguish excellent QA engineers



Best Practices in Test Execution — Professional QA Standards



01

Follow Test Cases Precisely

Execute every step exactly as documented. Do not improvise, combine steps, or skip actions. Consistency ensures reproducibility.

02

Validate Everything Thoroughly

After every action, confirm the result. Check all visible UI elements, data in database, logs, and network responses where applicable.

03

Record Detailed Evidence

Capture screenshots of every Pass, Fail, and Block. For failures, also capture browser console logs, network traffic, and server logs.

04

Communicate Issues Early

Don't silently wait when something is blocked or unclear. Raise flags immediately via Slack/Teams, daily standups, and JIRA comments.

05

Maintain Test Data Hygiene

Clean up test data after each cycle. Never leave orphaned or dirty data that could affect subsequent test runs.

06

Log Defects Same Day

Never accumulate defects for batch logging. Log each defect as soon as it is found to maintain real-time quality visibility.

07

Track Progress Continuously

Update the test management tool after every executed test case — not at the end of the day. Real-time progress tracking is essential.

08

Collaborate with Developers

Discuss complex defects with developers immediately. Early collaboration reduces time between defect logging and resolution.



Test Execution in Agile

How test execution fits into Agile sprints and daily QA activities



Test Execution in Agile — Sprint-Based Testing Activities



In Agile, test execution is continuous, iterative, and integrated into every 2-week sprint. QA engineers work alongside developers throughout — not after them.

TYPICAL AGILE SPRINT EXECUTION TIMELINE (2-WEEK SPRINT)



DAILY QA ACTIVITIES IN AGILE SPRINTS

- Morning standup: report execution progress, blockers, risks
- Execute prioritized test cases from the sprint backlog
- Log defects immediately; collaborate with devs on fixes
- Update test management tool with results in real time

CONTINUOUS TESTING PRINCIPLES

- Test early and often — don't wait for full feature completion
- Shift-left testing: QA reviews requirements, not just code
- Definition of Done: all test cases passed before story closes
- Automate regression suite grows with each sprint



Test Execution in CI/CD

Automated test execution in continuous integration and deployment pipelines



Test Execution in CI/CD Pipelines — Automated Quality Gates



In CI/CD, test execution is automated and triggered on every code commit or merge. Tests act as quality gates — the pipeline fails if tests fail.

CI/CD PIPELINE WITH INTEGRATED TEST EXECUTION



▲ QA Automation Touchpoints

CONTINUOUS INTEGRATION (CI)

- Every code commit triggers a full automated test run
- Developers get test results within minutes of committing code
- Failed tests block the merge — no broken code proceeds
- Tools: Jenkins, GitHub Actions, GitLab CI, Azure Pipelines

CONTINUOUS DEPLOYMENT (CD)

- Passed builds are automatically deployed to staging or prod
- Post-deployment smoke tests verify the live environment
- Failed post-deployment tests trigger automatic rollback
- QA role: maintain automation suite and review pipeline reports



Test Reporting After Execution

How to generate and communicate execution reports professionally



Test Reporting After Execution — Report Types & Audiences



Daily Execution Status Report

Audience: QA Lead, Project Manager

Frequency: Daily during execution

Tests executed today / Pass / Fail / Blocked counts, new defects logged, blockers, plan for tomorrow

Defect Summary Report

Audience: Development Team, QA Lead

Frequency: Daily / On demand

All open defects by severity, priority, and status. Defects fixed today, pending fixes, defect aging report

Test Execution Progress Report

Audience: Project Manager, Stakeholders

Frequency: Every 2-3 days

Execution progress percentage, cumulative pass/fail trend, coverage metrics, risk status, projection to completion

Test Summary Report

Audience: All Stakeholders, Management

Frequency: End of execution cycle

Final metrics, total defects found/fixed, quality assessment, go/no-go recommendation, risk register

Sample Test Execution Summary Report Template



This template is completed by the QA Lead at the end of each execution cycle and distributed to all stakeholders.

TEST EXECUTION SUMMARY REPORT — OrangeHRM v2.4.2 | Date: 2024-07-20 | QA Engineer: Sipho Dlamini

Metric	Value	Metric	Value
Total Test Cases Planned	80	Build Version Tested	v2.4.2
Total Test Cases Executed	80	Test Execution Period	2024-07-10 to 2024-07-20
Test Cases Passed	68 (85%)	Number of Defects Logged	16
Test Cases Failed	8 (10%)	Critical Defects Resolved	3/3 (100%)
Test Cases Blocked	4 (5%)	High Defects Resolved	7/8 (87.5%)
Pass Rate	85%	Defects Still Open	2 (Minor)
Requirements Coverage	92%	Go/No-Go Recommendation	CONDITIONAL GO

RISKS & NOTES

2 minor defects remain open (BUG-110, BUG-112). Risk accepted by PM — to be fixed in next hotfix release.

SIGN-OFF

QA Lead: _____ PM: _____ acceptable Date: _____



Role of QA Engineers During Execution

The professional responsibilities every QA engineer must own



Role of QA Engineers During Test Execution — Full Responsibilities



▶ Execute Test Cases

Systematically run all assigned test cases in priority order. Follow documentation precisely. Execute both manual and automated tests.

Identify and Report Defects

Detect deviations between expected and actual results. Immediately log all defects with complete, professional documentation in JIRA.

✓ Validate Defect Fixes

Perform retesting for all resolved defects. Confirm fixes are working in the correct build version. Update JIRA status accordingly.

Track and Report Progress

Maintain real-time updates in the test management tool. Provide daily progress reports to the QA Lead and project manager.

⚡ Communicate Risks Early

Proactively escalate blockers, environment issues, and quality risks. Never wait until the end of the sprint to raise concerns.

★ Ensure Quality Standards

Uphold the agreed quality gates — do not mark tests as Pass unless the expected result is fully met. Maintain professional integrity.



Why Strong Test Execution Makes Better QA Engineers

How mastering execution builds lasting professional skills



Strong Test Execution Builds Exceptional QA Engineers



Every test execution cycle is an opportunity to sharpen the skills that define great QA engineers — beyond just finding bugs.

Analytical Thinking

Dissecting complex user flows, identifying root causes of defects, and understanding system behavior patterns.

Impact: You become a problem solver — not just a button clicker.

Attention to Detail

Carefully comparing expected vs actual results, noting subtle UI differences, and reading error messages precisely.

Impact: You catch defects others miss — and earn trust from your team.

Defect Detection Ability

Developing an intuition for where software breaks — edge cases, integrations, data boundaries, and performance thresholds.

Impact: Your defect detection rate improves with every cycle you complete.

Technical Understanding

Working with tools like Selenium, Playwright, JIRA, TestRail, browsers, logs, and developer environments builds deep technical fluency.

Impact: You bridge the gap between QA and development — making you invaluable.

Professional Communication

Writing precise defect reports, daily status updates, and clear risk communications that stakeholders and developers trust.

Impact: Strong communication accelerates defect resolution and career growth.

Key Takeaways — Test Execution

01

Test Execution is the STLC phase where test cases are run, results compared, and defects identified — it is where quality is measured.

02

The 10-step workflow (Receive Build → Generate Reports) provides a structured approach every QA professional must follow without exception.

03

Manual and Automated execution are complementary — automation (Selenium, Playwright) handles regression at speed; manual handles exploration.

04

Every result must be documented with evidence. Every defect must be logged with complete reproduction steps, severity, and priority.

05

Retesting confirms fixes. Regression testing ensures fixes didn't break anything else. Both are mandatory after every defect resolution.

06

In Agile and CI/CD, execution is continuous — QA engineers test every sprint and automation runs in every pipeline as a quality gate.

07

Strong test execution builds analytical thinking, technical skill, attention to detail, and communication — the foundation of a QA career.